

# Traffic Anomaly Detection

Deteksi anomali volume trip Jakarta berdasarkan kondisi cuaca real-time, dibangun di atas Apache Flink (PyFlink Table API + DataStream API), Kafka, dan MySQL — proyek belajar/hackathon [/home/theo/flink](#) .

## 1 Overview

Use case ini menggabungkan dua stream Kafka — `trip_events` (data perjalanan/trip) dan `weather_events` (data cuaca) — lewat Apache Flink, untuk dua tujuan berbeda yang saling melengkapi:

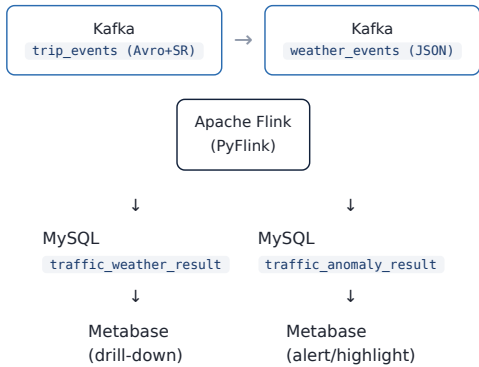
### PIPELINE 1 — ENRICHMENT

Tiap trip individual "dikasih" konteks cuaca saat terjadinya. Hasilnya tabel mentah per-trip yang bisa dieksplorasi bebas (drill-down, ad-hoc analysis).

### PIPELINE 2 — ANOMALY DETECTION

Trip diagregasi per zona & per window 15 menit, dibandingkan ke ekspektasi berdasar cuaca. Hasilnya sinyal siap-pakai: `is_anomaly = true/false`.

### Arsitektur end-to-end



Kedua job Flink membaca topic Kafka yang sama secara independen (beda `group.id`: `pyflink-interval-join-demo` vs `pyflink-traffic-anomaly`), jadi bisa jalan bersamaan tanpa saling ganggu.

## 2 Contoh Data Sumber (Topic Kafka)

Dua topic dengan format berbeda secara sengaja (meniru desain nyata: `trip_events` datang dari CDC Debezium sehingga Avro+Schema Registry, `weather_events` dari producer sederhana sehingga JSON polos):

### Topic `trip_events` (Avro + Confluent Schema Registry)

| id   | pickup_datetime  | pu_location_id | pu_location_name | trip_distance | fare_amount |
|------|------------------|----------------|------------------|---------------|-------------|
| 1025 | 2024-01-15 17:03 | 7              | Grogol           | 1.1           | 8           |
| 1031 | 2024-01-15 17:15 | 13             | Menteng          | 4.5           | 19          |
| 1042 | 2024-01-15 18:10 | 35             | Cawang           | 7.8           | 32          |

### Topic `weather_events` (plain JSON)

| location_id       | event_time       | condition  | precipitation (mm) |
|-------------------|------------------|------------|--------------------|
| 3 (Jakarta Barat) | 2024-01-15 17:00 | clear      | 0                  |
| 1 (Jakarta Pusat) | 2024-01-15 17:10 | heavy_rain | 22.5               |

#### KENAPA GRANULARITAS LOKASINYA BEDA

`trip_events` dikirim per **kelurahan** (10 zona: Kuningan, Tomang, Kebon Jeruk, Menteng, Cikini, Kelapa Gading, Pluit, Kebayoran Baru, Kemang, Cawang), sementara `weather_events` cuma per **kota administrasi** (5 zona BMKG: Jakarta Pusat/Utara/Barat/Selatan/Timur) — di dunia nyata tidak ada 1 stasiun cuaca per kelurahan. Trip di-lookup dulu ke zona cuacanya lewat UDF `weather_zone_of(pu_location_id)` sebagai *computed column*, bukan JOIN biasa ke tabel referensi — supaya kolom waktu `pickup_datetime` tetap sah sebagai *time attribute* (dibutuhkan interval join di bawah).

## 3 Pipeline 1 — Interval Join (Enrichment)

`jobs/hackatown/flink_sql_interval_join.py`

### Kondisi join

```
FROM trip_events t
JOIN weather_events w
  ON t.weather_location_id = w.location_id
  AND t.pickup_datetime BETWEEN w.event_time - INTERVAL '30' MINUTE
                           AND w.event_time + INTERVAL '30' MINUTE
```

Hasil join dari contoh data di atas

| id             | zona cuaca | pickup | weather terdekat       | ±30 menit? | hasil                |
|----------------|------------|--------|------------------------|------------|----------------------|
| 1025 (Grogol)  | 3          | 17:03  | 17:00, clear           | ya         | ✓ match              |
| 1031 (Menteng) | 1          | 17:15  | 17:10, heavy_rain      | ya         | ✓ match              |
| 1042 (Cawang)  | 5          | 18:10  | tidak ada event zona 5 | —          | ✗ hilang dari output |

CATATAN PENTING

Ini **INNER JOIN** — trip yang zonanya belum kebagian weather event dalam ±30 menit (baris 1042 di atas) **diam-diam hilang** dari output, bukan muncul dengan `weather_condition = NULL` . Perlu diingat kalau baca angka `trip_count` di dashboard — bisa under-count.

Baris yang masuk ke `traffic_weather_result` (MySQL)

| id   | pickup_datetime | pu_location_id | pu_location_name | weather_location_id | trip_distance | fare_amount | weather_condition | precipitation |
|------|-----------------|----------------|------------------|---------------------|---------------|-------------|-------------------|---------------|
| 1025 | 17:03           | 7              | Grogol           | 3                   | 1.1           | 8           | clear             | 0             |
| 1031 | 17:15           | 13             | Menteng          | 1                   | 4.5           | 19          | heavy_rain        | 22.5          |

## 4 Pipeline 2 — Anomaly Detection

`jobs/hackatown/flink_sql_traffic_anomaly.py` — file baru, **tidak mengubah** pipeline 1 sama sekali (source DDL & skema pesan Kafka identik, sengaja terduplikasi bukan di-import, supaya use case lama tetap persis seperti semula).

### Kenapa butuh Table API + DataStream API sekaligus

Interval join wajib Table API/SQL (DataStream API PyFlink tidak punya operator interval-join siap pakai). Tapi window 15 menit + custom logic per zona butuh `ProcessWindowFunction`, yang cuma ada di DataStream API. Jadi hasil join "diturunkan" ke DataStream ( `to_data_stream()` ), diproses per window, lalu "dinaikkan" lagi ke Table ( `from_data_stream()` ) untuk di-sink ke MySQL.

### Logic `AnomalyDetector` — per window 15 menit, per zona cuaca

- `trip_count` = jumlah trip yang ke-join dalam window itu.
- `weather_condition` dominan = modus kondisi cuaca di window itu (cuaca bisa berubah dalam 15 menit).
- Lookup `expected_trip_count` dari tabel ekspektasi (dikalibrasi dari parameter simulasi producer, baseline 36 trip/window saat cuaca *clear*).
- `is_anomaly = True` kalau `trip_count` di luar rentang  $[\text{expected} \times 0.5, \text{expected} \times 1.5]$ .

### Tabel ekspektasi (baseline)

| Kondisi cuaca | Expected trip/window | Rentang wajar ( $\times 0.5 - \times 1.5$ ) |
|---------------|----------------------|---------------------------------------------|
| clear         | 36                   | 18 - 54                                     |
| cloudy        | 31                   | 15.5 - 46.5                                 |
| light_rain    | 22                   | 11 - 33                                     |
| heavy_rain    | 11                   | 5.5 - 16.5                                  |
| storm         | 5                    | 2.5 - 7.5                                   |

### Contoh skenario

| Window                     | Cuaca dominan | trip_count | Rentang wajar | Hasil                                            |
|----------------------------|---------------|------------|---------------|--------------------------------------------------|
| Jakarta Barat, 17:00–17:15 | clear         | 40         | 18–54         | normal                                           |
| Jakarta Pusat, 17:15–17:30 | heavy_rain    | 10         | 5.5–16.5      | normal — trip turun karena hujan itu diharapkan  |
| Jakarta Pusat, 17:30–17:45 | heavy_rain    | 35         | 5.5–16.5      | ANOMALI — harusnya turun, malah 2x di atas batas |
| Jakarta Timur, storm       | storm         | 1          | 2.5–7.5       | ANOMALI — di bawah minimum wajar                 |

#### POIN KUNCI

Trip sepi saat hujan **bukan** anomali — itu pola yang memang disengaja lewat bias di producer ( `EMIT_PROBABILITY_BY_CONDITION` ). Yang ditandai anomali cuma yang **menyimpang dari pola cuaca yang berlaku saat itu**, dua arah (naik maupun turun ekstrem).

#### KETERBATASAN

Baseline masih statis (dihitung manual dari parameter simulasi, bukan histori data asli). Window yang benar-benar kosong (0 trip) tidak pernah diproses sama sekali — window keyed di Flink baru "hidup" begitu ada minimal 1 elemen — jadi "zona mati total" tidak akan pernah ke-flag lewat mekanisme ini.

## 5 Perbandingan Kedua Tabel Hasil

| Aspek                  | <code>traffic_weather_result</code> (Pipeline 1)                                                | <code>traffic_anomaly_result</code> (Pipeline 2)                                     |
|------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Granularitas           | Per-trip individual                                                                             | Per-zona, per window 15 menit                                                        |
| Tujuan                 | Enrichment — bahan mentah eksplorasi                                                            | Detection — sinyal siap pakai                                                        |
| Kolom kunci            | <code>pu_location_id/name</code> , <code>trip_distance</code> , <code>precipitation</code> (mm) | <code>trip_count</code> , <code>expected_trip_count</code> , <code>is_anomaly</code> |
| Pertanyaan dijawab     | "Cuaca apa saat trip ini terjadi?"                                                              | "Apakah ini wajar mengingat cuacanya?"                                               |
| Cara pakai di Metabase | Drill-down, filter manual, scatter/histogram                                                    | Filter <code>is_anomaly = true</code> , langsung kelihatan                           |
| Kelemahan              | Harus bikin sendiri logic "wajar/tidak"-nya                                                     | Kehilangan detail per-kelurahan & per-trip                                           |

Kombinasi keduanya: dashboard anomali dipakai buat **highlight masalahnya** (kapan & di mana), lalu drill-down ke tabel enrichment mentah buat **investigasi akar penyebabnya** (trip individual mana yang berkontribusi).

## 6 Status & Verifikasi

| Komponen                                                                                                         | Status                                                                      |
|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| DDL parsing, UDF lookup, interval join, bridging Table↔DataStream (window + <code>ProcessWindowFunction</code> ) | ✓ <b>terverifikasi jalan</b> (tanpa exception, job graph berhasil dibangun) |

|                                                                     |                                                                               |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Baca data Avro/JSON asli dari broker Kafka dev Bank Sinarmas        | <b>belum terverifikasi</b> — broker tidak reachable dari sandbox pengembangan |
| Tulis ke MySQL beneran ( <code>kafka-db/docker-compose.yml</code> ) | <b>belum terverifikasi end-to-end</b>                                         |
| Dashboard Metabase                                                  | belum dihubungkan ke tabel hasil                                              |